

Module 10 | From Computational Graph to Autograd Implementation

Problem Statement (Rephrased from [Practice Problems](#))

SECTION A: Conceptual Understanding

1. Why do we need **autograd** if we already understand calculus and can compute derivatives manually?
2. What problem does autograd solve when working with **deep neural networks containing many layers and parameters**?
3. What is a **computational graph**? Explain it using the simple expression:

$$y = (x^2 + 3x) \times 2$$

4. In a computational graph:
 - What do the **nodes** represent?
 - What do the **edges** represent?
5. Why is a computational graph called a **Directed Acyclic Graph (DAG)**?
6. What would happen to gradient flow if the computational graph contained a **cycle**?
7. In your own words, explain how **autograd automatically applies the chain rule** to compute gradients.

SECTION B: Computational Graph and Chain Rule

8. Consider:

$$x = 2$$

$$y = x^2$$

$$z = 3y$$

$$L = z + 4$$

Tasks:

- Draw the computational graph.
- Manually compute $\frac{dL}{dx}$ using the chain rule.

9. Suppose:

$$y = \text{sigmoid}(wx + b)$$

and

$$L = (y - t)^2$$

Write the **complete chain-rule expression** for $\frac{dL}{dw}$ before simplifying it.

10. Why does the combination of **Sigmoid activation + Binary Cross-Entropy loss** simplify the gradient to:

$$\frac{\partial L}{\partial z} = y_{\text{pred}} - y$$

11. What would change in the **gradient flow and gradient computation** if we used **ReLU** instead of **Sigmoid**?

My Solution

Q01. Why do we need **autograd** if we already understand calculus and can compute derivatives manually?

Suppose, I am an Automotive Engineer. If I want to go to Netrakona from Cumilla to visit my friend, either I should take a public bus which I didn't build or I should build a bus, which is a task that comes with a lot of challenges.

Not using Autograd provided by PyTorch is equivalent to **Building my own Bus**.

Challenges of Building Your Own Autograd with PyTorch

Building an automatic differentiation system like PyTorch's `autograd` is much harder than simply calculating derivatives. Some of the main challenges are:

1. Correctness of Reverse-Mode Automatic Differentiation

- You must correctly calculate the **vector-Jacobian product (VJP)** for every operation.
- Operations such as **broadcasting**, **reductions**, and **views** can easily lead to subtle gradient errors.
- **In-place operations** may overwrite values that are still needed during the backward pass.
- Special cases such as `None` gradients, zero gradients, and `NaN` values require careful handling.

2. Dynamic Computation Graph Management

- The computation graph must usually be **rebuilt during every forward pass** so that Python control flow, such as `if` statements and loops, can be handled correctly.
- Operations whose shapes or behavior depend on the input data make graph management more complicated.
- Supporting **higher-order derivatives** (derivatives of derivatives) greatly increases both the implementation complexity and memory requirements.

3. Memory Issues

- Intermediate values needed for backpropagation must remain in memory until the backward pass is completed.
- Poorly managed references can create **reference cycles** and memory leaks.
- Long computation graphs or large neural networks can quickly consume large amounts of memory and cause **out-of-memory (OOM)** errors.

4. Performance Limitations

- If the computation graph is implemented mainly in Python, each graph operation introduces significant **Python interpreter overhead**.
- You would not have access to PyTorch's huge collection of highly optimized operators and **fused kernels**.
- Efficient **GPU utilization** and scaling are difficult to achieve compared with PyTorch's optimized C++/CUDA engine.

5. Integration Difficulties

Even if your `autograd` system calculates gradients correctly, making it behave like PyTorch's `autograd` is another major challenge.

- You would need to reproduce PyTorch's behavior for `.grad`, **leaf tensors**, **hooks**, and **optimizers**.
- Making your system work with components such as `nn.Module`, **Automatic Mixed Precision (AMP)**, `torch.compile`, and **distributed training** is difficult.

- You would need extensive **numerical testing** to verify that your gradients match the results produced by PyTorch's real `autograd`.

Key Idea:

Building a small `autograd` engine is a great way to understand backpropagation. Building something as complete, reliable, and fast as PyTorch's `autograd` is a much bigger engineering problem.

Q02. What problem does `autograd` solve when working with deep neural networks containing many layers and parameters?

`Autograd` solves the problem of manually computing and managing a huge number of derivatives in a deep neural network.

A deep neural network may contain:

- Many **layers**.
- Thousands, millions, or even billions of **parameters**.
- Complex mathematical operations connecting those parameters.
- Long chains of dependencies between the parameters, intermediate values, and final loss.

To train the network, we need the gradient of the loss with respect to **every parameter**:

$$\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n}$$

Computing all of these derivatives manually would be extremely difficult and error-prone.

Autograd automatically:

1. Builds a **computation graph** during the forward pass.
2. Records the operations performed on tensors.
3. Applies the **chain rule** during the backward pass.
4. Computes the gradients of the loss with respect to the network's parameters.
5. Stores those gradients so that an optimizer such as SGD or Adam can update the parameters.

For example:

$$x \rightarrow \text{Layer 1} \rightarrow \text{Layer 2} \rightarrow \text{Layer 3} \rightarrow L$$

Instead of manually deriving:

$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial W_3},$$

we can simply compute:

```
loss.backward()
```

and PyTorch's `autograd` performs the required backward computation automatically.

In short: Autograd turns the difficult task of manually deriving and calculating thousands or millions of interconnected gradients into an automatic backward pass through the computation graph.

Q03. What is a **computational graph**? Explain it using the simple expression: $y = (x^2 + 3x) \times 2$.

A **computational graph** is a way of representing a mathematical computation as a **graph of values and operations**.

It shows:

- **What values are involved** in the computation.
- **What operations are performed** on those values.
- **How the output of one operation becomes the input to another operation.**

Consider the expression:

$$y = (x^2 + 3x) \times 2$$

We can break the computation into smaller steps:

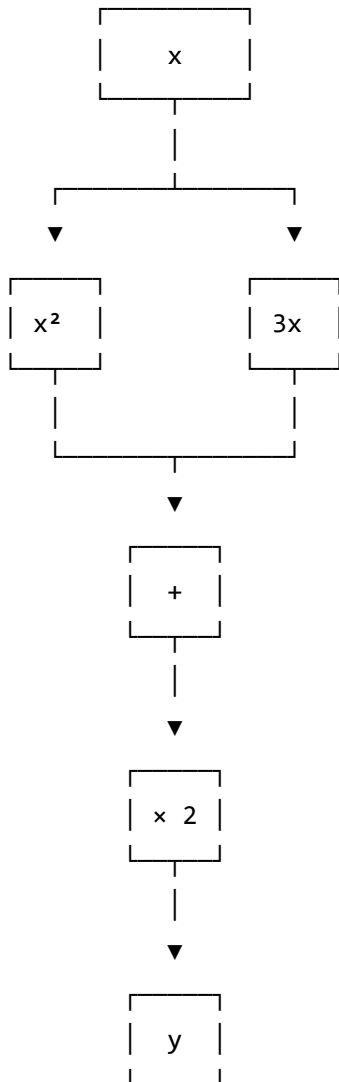
$$a = x^2$$

$$b = 3x$$

$$c = a + b$$

$$y = c \times 2$$

The corresponding computational graph can be represented as:



Here:

- x is the **input value**.
- x^2 and $3x$ are **operations** applied to x .
- $+$ adds the two resulting values.
- $\times 2$ multiplies the result by 2.
- y is the **final output**.

So the graph describes the complete path of the computation:

$$x \rightarrow x^2; 3x \rightarrow x^2 + 3x \rightarrow (x^2 + 3x) \times 2 \rightarrow y$$

Q04. In a computational graph, what do the **nodes** represent? What do the **edges** represent?

In a computational graph:

- **Nodes** represent the values and operations involved in a computation.
For example, nodes can represent input tensors, intermediate values, parameters, or operations such as addition and multiplication.
- **Edges** represent the flow of data between the nodes.
They show how the output of one operation becomes the input to another operation.

Q05. Why is a computational graph called a **Directed Acyclic Graph (DAG)**?

Yes. A computational graph is naturally a **Directed Acyclic Graph (DAG)** in the usual forward computation.

- **Directed** — The edges have a direction. They show how data flows from inputs toward the final output.
- **Acyclic** — There are no cycles in the graph. Once you move forward through the computation, you cannot return to a previous node.

For example:

$$x \rightarrow x^2 \rightarrow x^2 + 3x \rightarrow (x^2 + 3x) \times 2 \rightarrow y$$

The computation always moves **forward**:

Input → Operation → Intermediate Value → Operation → Output

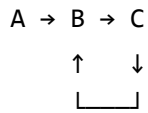
There is no path such as:

A → B → C → A

Q06. What would happen to gradient flow if the computational graph contained a **cycle**?

If a computational graph contained a **cycle**, gradient flow would become problematic because the graph would no longer be a **Directed Acyclic Graph (DAG)**.

For example:



Here, B depends on C , while C also depends on B . This creates a circular dependency.

During the backward pass, autograd normally starts from the loss and moves backward through the graph. With a cycle, it could encounter the same nodes repeatedly:

$\text{Loss} \rightarrow C \rightarrow B \rightarrow C \rightarrow B \rightarrow C \rightarrow \dots$

There would be no natural point at which the backward traversal could finish.

This creates several problems:

- **Gradient computation may not terminate** because the backward traversal can keep following the cycle.
- The **chain rule becomes recursively dependent** on values or gradients that depend on themselves.
- The system cannot use the simple **topological ordering** that makes ordinary backpropagation efficient.
- Memory usage could grow as the system attempts to keep track of the repeated dependencies.

For example, if:

$$y = f(x)$$

but x somehow depended on y :

$$x = g(y)$$

then we would have:

$$x \rightarrow y \rightarrow x \rightarrow y \rightarrow \dots$$

This is fundamentally different from the ordinary feed-forward computation used by a standard computational graph.

In short: A cycle creates a circular dependency, so the normal forward and backward traversal of a computational graph breaks down. This is why computational graphs used by ordinary reverse-mode autograd are structured as **DAGs**.

Q07. In your own words, explain how **autograd** automatically applies the chain rule to compute gradients.

Think of **autograd** as a system that remembers how a result was computed and then walks backward through those computations.

Suppose we have:

$$y = f(g(x))$$

The chain rule tells us:

$$\frac{dy}{dx} = \frac{dy}{dg} \frac{dg}{dx}$$

Autograd does this automatically in roughly three steps:

1. **During the forward pass**, autograd records the operations that produced each value and builds a computation graph.

$$x \rightarrow g(x) \rightarrow y$$

2. **During the backward pass**, it starts from the final result, usually the loss, and moves backward through the graph.

$$x \leftarrow g(x) \leftarrow y$$

3. At each operation, it calculates the **local derivative** and multiplies it by the gradient that has already arrived from the next operation.

This is exactly the **chain rule**.

For example, if:

$$z = g(x)$$

and:

$$y = f(z)$$

then during backpropagation:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial x}$$

Autograd computes these pieces and combines them as it moves backward through the graph.

For a deep neural network, the same idea is repeated across many layers:

Forward:

Input $\rightarrow$ Layer 1 $\rightarrow$ Layer 2 $\rightarrow$ Layer 3 $\rightarrow$ Loss

Backward:

Input $\leftarrow$ Layer 1 $\leftarrow$ Layer 2 $\leftarrow$ Layer 3 $\leftarrow$ Loss

$\uparrow$
 gradients flow

So, **autograd does not use a different rule for differentiation**. It uses the ordinary chain rule repeatedly, while automatically keeping track of the operations and combining their local derivatives in the correct order.

Q08. Consider: $x = 2$, $y = x^2$, $z = 3y$ and $L = z + 4$. Draw the computational graph. Manually compute $\frac{dL}{dx}$ using the chain rule.

The Computational Graph:

$$x \rightarrow \square^2 \rightarrow y \rightarrow 3 \times \square \rightarrow z \rightarrow \square + 4 \rightarrow L$$

$$\begin{aligned} \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x} \\ &= 1 \cdot 3 \cdot 2x \\ &= 6x \end{aligned}$$

$$\begin{aligned} \left(\frac{\partial L}{\partial x} \right)_{x=2} &= 6 \times 2 \\ &= 12 \end{aligned}$$

Q09. Suppose $y = \text{sigmoid}(wx + b)$, and $L = (y - t)^2$, write the complete chain-rule expression for $\frac{dL}{dw}$ before simplifying it.

$$\begin{aligned}\frac{\partial L}{\partial w} &= \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial (wx + b)} \cdot \frac{\partial (wx + b)}{\partial w} \\ &= 2(y - t) \cdot \sigma'(wx + b) \cdot (1 - \sigma'(wx + b)) \cdot x \\ &= 2(y - t) \cdot y(1 - y) \cdot x \\ &= -2xy(y - 1)(y - t)\end{aligned}$$

Q10. Why does the combination of Sigmoid activation + Binary Cross-Entropy loss simplify the gradient to: $\frac{\partial L}{\partial z} = y_{\text{pred}} - y$.

The simplification happens because the **derivative of the sigmoid function cancels with part of the derivative of the Binary Cross-Entropy (BCE) loss**.

Let:

$$y_{\text{pred}} = \sigma(z)$$

where the sigmoid function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The BCE loss for one example is:

$$L = -[y \log(y_{\text{pred}}) + (1 - y) \log(1 - y_{\text{pred}})]$$

We want:

$$\frac{\partial L}{\partial z}$$

Because the loss depends on z through y_{pred} , we use the **chain rule**:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y_{\text{pred}}} \cdot \frac{\partial y_{\text{pred}}}{\partial z}$$

Step 1: Derivative of BCE with respect to y_{pred}

Starting with:

$$L = -[y \log(y_{\text{pred}}) + (1 - y) \log(1 - y_{\text{pred}})]$$

Differentiate:

$$\frac{\partial L}{\partial y_{\text{pred}}} = -\frac{y}{y_{\text{pred}}} + \frac{1 - y}{1 - y_{\text{pred}}}$$

Combining the terms:

$$\frac{\partial L}{\partial y_{\text{pred}}} = \frac{y_{\text{pred}} - y}{y_{\text{pred}}(1 - y_{\text{pred}})}$$

Step 2: Derivative of Sigmoid

For:

$$y_{\text{pred}} = \sigma(z)$$

the derivative is:

$$\frac{\partial y_{\text{pred}}}{\partial z} = y_{\text{pred}}(1 - y_{\text{pred}})$$

Step 3: Apply the chain rule

Now multiply the two derivatives:

$$\frac{\partial L}{\partial z} = \frac{y_{\text{pred}} - y}{y_{\text{pred}}(1 - y_{\text{pred}})} \cdot y_{\text{pred}}(1 - y_{\text{pred}})$$

The common terms cancel:

$$\boxed{\frac{\partial L}{\partial z} = y_{\text{pred}} - y}$$

So the complicated-looking expression becomes remarkably simple.

Q11. What would change in the **gradient flow and gradient computation** if we used **ReLU** instead of **Sigmoid**?

If we replace **Sigmoid** with **ReLU**, the gradient computation changes significantly.

For Sigmoid:

$$y_{\text{pred}} = \sigma(z)$$

and:

$$\frac{dy_{\text{pred}}}{dz} = y_{\text{pred}}(1 - y_{\text{pred}})$$

When Sigmoid is combined with Binary Cross-Entropy, these terms cancel through the chain rule, giving:

$$\frac{\partial L}{\partial z} = y_{\text{pred}} - y$$

With **ReLU**:

$$a = \text{ReLU}(z) = \max(0, z)$$

Its derivative is:

$$\frac{da}{dz} = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$

So during backpropagation, the gradient behaves differently:

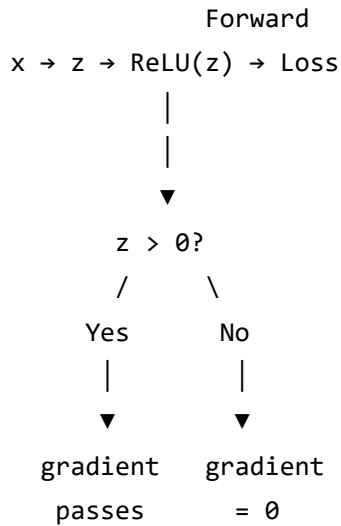
- When $z > 0$, ReLU passes the incoming gradient backward **unchanged**:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}$$

- When $z < 0$, ReLU blocks the gradient:

$$\frac{\partial L}{\partial z} = 0$$

Conceptually:



This means ReLU does **not** produce the same simple gradient $y_{\text{pred}} - y$ that we get from Sigmoid + BCE.

Instead, the gradient follows the chain rule:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z}$$

and therefore:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}$$

The important difference in **gradient flow** is that Sigmoid has a smooth derivative that can become very small, especially when the activation is near 0 or 1. ReLU, on the other hand, has a derivative of exactly **1 for positive inputs**, so it allows gradients to flow backward strongly through those neurons.

However, for negative inputs, ReLU's derivative is **0**, so the neuron receives no gradient through that path. If a neuron consistently receives negative inputs, it can become a **dead ReLU** and stop learning.